

Table of contents

Métaheuristiques d'optimisation : PSO, Firefly Algorithm, Cuckoo Search	1
Particle Swarm Optimization (PSO)	1
Introduction	1
Algorithme PSO	1
Firefly Algorithm	2
Introduction	2
Règle de déplacement	2
Cuckoo Search	3
Introduction	3
Algorithme	3

Métaheuristiques d'optimisation : PSO, Firefly Algorithm, Cuckoo Search

Particle Swarm Optimization (PSO)

Introduction

La méthode d'optimisation par essaim particulaire (PSO) a été introduite par Eberhart & Kennedy en 1995. Inspirée du comportement collectif d'animaux sociaux (comme les vols d'oiseaux ou les bancs de poissons), elle est adaptée aux problèmes d'optimisation continue.

Principes fondamentaux

- C'est une **métaheuristique basée sur une population** d'agents (particules).
- Chaque particule explore l'espace de recherche et ajuste sa trajectoire en fonction de deux expériences :
 - Son **expérience personnelle** (meilleure position qu'elle a trouvée).
 - L'**expérience sociale** (meilleure position trouvée par l'ensemble de l'essaim).
- Un troisième facteur, l'**inertie**, préserve une partie de la vitesse précédente.

Algorithme PSO

On initialise un essaim de N particules, chacune ayant une position $\mathbf{x}_i(t)$ et une vitesse $\mathbf{v}_i(t)$ à l'itération t .

Mise à jour de la vitesse et de la position

La vitesse est mise à jour selon trois composantes :

$$\mathbf{v}_i(t+1) = \omega \mathbf{v}_i(t) + c_1 r_1(t+1) [\mathbf{x}_i^{\text{best}}(t) - \mathbf{x}_i(t)] + c_2 r_2(t+1) [\mathbf{B}(t) - \mathbf{x}_i(t)]$$

Puis la position est mise à jour :

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1)$$

Paramètres guidés (guided parameters)

- ω (**coefficient d'inertie**) : Contrôle la persistance de la vitesse précédente. Typiquement légèrement inférieur à 1 (p. ex. 0.9).
- c_1 (**coefficient cognitif**) : Pondere l'attraction vers la meilleure position personnelle (expérience individuelle). Classiquement $c_1 \approx 2$.
- c_2 (**coefficient social**) : Pondere l'attraction vers la meilleure position globale (expérience collective). Classiquement $c_2 \approx 2$.
- r_1, r_2 : Nombres aléatoires uniformes dans $[0, 1[$ introduisant une **stochasticité**.

Remarques générales

- On limite souvent la vitesse ($v_{\max}$) et la position ($x_{\max}$) pour éviter des dépassements.
- Le PSO converge généralement rapidement.
- Il existe des variantes pour problèmes combinatoires (arrondi, codages spécifiques).

Exemple synthétique

Pour un problème simple à une dimension, on observe comment deux particules s'attirent mutuellement vers l'optimum. La particule au voisinage de l'optimum devient le *global-best*, attirant l'autre particule, qui peut à son tour la dépasser et devenir le nouveau guide. Le processus itératif mène à une convergence rapide vers la région optimale.

Firefly Algorithm

Introduction

L'algorithme des lucioles (Firefly Algorithm), introduit par Xin-She Yang (2009), s'inspire du comportement de bioluminescence des lucioles pour attirer des partenaires ou des proies.

Principes de base

- Chaque luciole i représente une solution candidate x_i dans l'espace de recherche.
- L'intensité lumineuse I_i d'une luciole est proportionnelle à la qualité (fitness) de sa solution :
 $I_i(t) = f(x_i(t))$ pour une maximisation.
- Les lucioles moins brillantes sont attirées par les plus brillantes.

Règle de déplacement

La luciole i (moins brillante) se déplace vers la luciole j (plus brillante) selon :

$$x_i^{\text{new}} = x_i + \beta_0 \exp(-\gamma r_{ij}^2)(x_j - x_i) + \alpha \epsilon$$

Paramètres guidés

- β_0 : Attractivité de base (souvent fixée à 1).
- γ : Coefficient d'absorption de la lumière. Il règle l'équilibre exploration/exploitation :
 - Petit γ : La lumière porte loin $\rightarrow$ attractivité des lucioles lointaines $\rightarrow$ favorise l'**exploration**.
 - Grand γ : La lumière est vite absorbée $\rightarrow$ attractivité forte vers les meilleures solutions $\rightarrow$ favorise l'**exploitation**.
- α : Paramètre de randomisation (typiquement $\alpha \in [0, 1]$).
- ϵ : Vecteur de bruit aléatoire (distribution uniforme, normale ou de Lévy).

Algorithme

1. Initialiser la population de lucioles.
2. Pour chaque paire (i, j) , si $I_i < I_j$, déplacer i vers j .
3. Mettre à jour les intensités.
4. Répéter jusqu'à convergence.

Exemple synthétique

Dans un espace de recherche 2D, plusieurs lucioles convergent progressivement vers l'optimum global. L'attractivité décroît exponentiellement avec la distance, ce qui permet à l'essaim d'explorer largement au début, puis de se concentrer (exploiter) autour des bonnes solutions.

Cuckoo Search

Introduction

Le Cuckoo Search, introduit par Yang & Deb (2010), s'inspire du parasitisme de couvée de certains coucous.

Principes de base

- **Nids** : Chaque nid héberge une solution candidate (un "œuf").
- **Parasitisme** : Un coucou dépose son œuf (nouvelle solution) dans un nid choisi au hasard.
- **Découverte** : L'oiseau hôte peut découvrir l'œuf étranger (avec probabilité p_a) et l'abandonner.

Algorithme

À chaque itération :

1. Générer une nouvelle solution (œuf) par un **vol de Lévy** (mécanisme d'exploration).
2. Choisir un nid j au hasard.
3. Si la nouvelle solution est meilleure que celle du nid j , la remplacer.
4. Avec une probabilité p_a , abandonner une fraction des pires nids et en créer de nouveaux aléatoirement (mécanisme d'exploitation).

Paramètres guidés

- p_a (**probabilité d'abandon**) : Contrôle la fraction de pires solutions qui sont remplacées à chaque itération. Typiquement $p_a \approx 0.25$.
- **Paramètres du vol de Lévy** : L'exposant α de la distribution de Lévy contrôle la longueur des sauts exploratoires.

Exemple synthétique

Pour maximiser une fonction à une variable avec 4 nids, on observe qu'une petite fraction des nids (les pires) est régulièrement remplacée par des solutions aléatoires. Ce processus permet une exploration soutenue tout en conservant et en améliorant les bonnes solutions. L'algorithme trouve souvent une solution proche de l'optimum global avec un effort de calcul moindre qu'une recherche exhaustive.